KenPOS - Kenyan Point of Sale System: Architecture Documentation

Version: 1.0
Last Updated: May 4, 2026
Project: Kenya POS (Manus WebDev)

Table of Contents

1. Executive Overview
2. Technology Stack
3. System Architecture
4. Database Schema
5. Backend Architecture
6. Frontend Architecture
7. API Design & tRPC Procedures
8. Frontend Pages & Features
9. Authentication & Authorization
10. Key Features & Workflows
11. Deployment & Environment

Executive Overview

KenPOS is a comprehensive **Point of Sale (POS) system** designed for Kenyan restaurants, cafes, and retail establishments. It provides integrated sales transaction processing, inventory management, customer relationship management, staff payroll, and advanced reporting capabilities.

Core Objectives

- **Fast POS Operations:** Real-time sales transaction processing with multiple payment methods (cash, M-Pesa, Stripe, wallet)
- **Inventory Control:** Stock tracking, low-stock alerts, and category-based product management
- **Customer Management:** Loyalty programs, wallet balances, spending history, and credit accounts
- **Staff Management:** Role-based access control, activity logging, attendance tracking, and payroll processing
- **Business Intelligence:** Comprehensive sales reports, customer spending analysis, and transaction audits
- **Multi-Branch Support:** Manage multiple branches and serving points from a single system

Architecture Highlights

- **Full-Stack TypeScript:** Unified language across frontend and backend for type safety
- **tRPC + React Query:** End-to-end type-safe API with automatic client generation
- **Drizzle ORM:** Type-safe database layer with MySQL
- **React 19 + Tailwind CSS 4:** Modern UI with responsive design
- **Express.js:** Lightweight backend server with OAuth integration
- **Role-Based Access Control (RBAC):** Fine-grained permissions at both frontend and backend

---

Technology Stack

Frontend

Layer	Technology	Purpose
UI Framework	React 19	Component-based UI rendering
Styling	Tailwind CSS 4	Utility-first CSS framework
Component Library	shadcn/ui + Radix UI	Pre-built accessible components
Routing	Wouter	Lightweight client-side routing
State Management	React Query (@tanstack/react-query)	Server state management
API Client	tRPC + @trpc/react-query	Type-safe RPC calls
Forms	React Hook Form + Zod	Form validation and submission
Charts	Recharts	Data visualization
Icons	Lucide React	Icon library
PDF Export	jsPDF + pdfjs-dist	PDF generation and preview
Excel Export	XLSX	Excel file generation
Animations	Framer Motion	Smooth transitions
Build Tool	Vite	Fast development and production builds

Backend

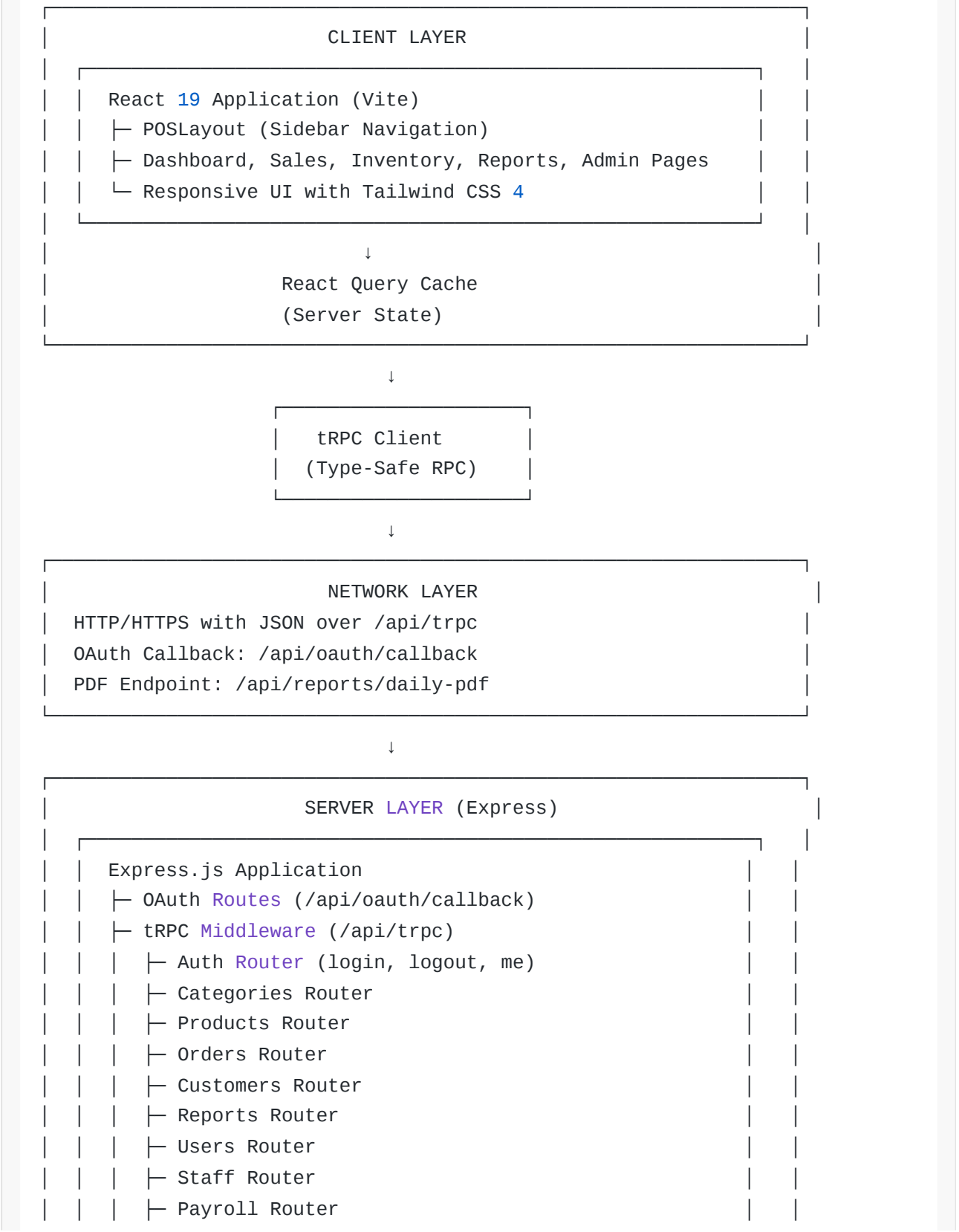
Layer	Technology	Purpose
Runtime	Node.js	JavaScript runtime
Framework	Express.js	HTTP server and middleware
API Protocol	tRPC	Type-safe RPC framework
Database	MySQL 8	Relational database
ORM	Drizzle ORM	Type-safe database queries
Authentication	Manus OAuth	OAuth 2.0 integration
PDF Generation	PDFKit	Server-side PDF creation
File Storage	AWS S3	Cloud file storage
Payment Processing	Stripe + M-Pesa	Payment gateway integration
Testing	Vitest	Unit testing framework
Type Checking	TypeScript 5.9	Static type checking

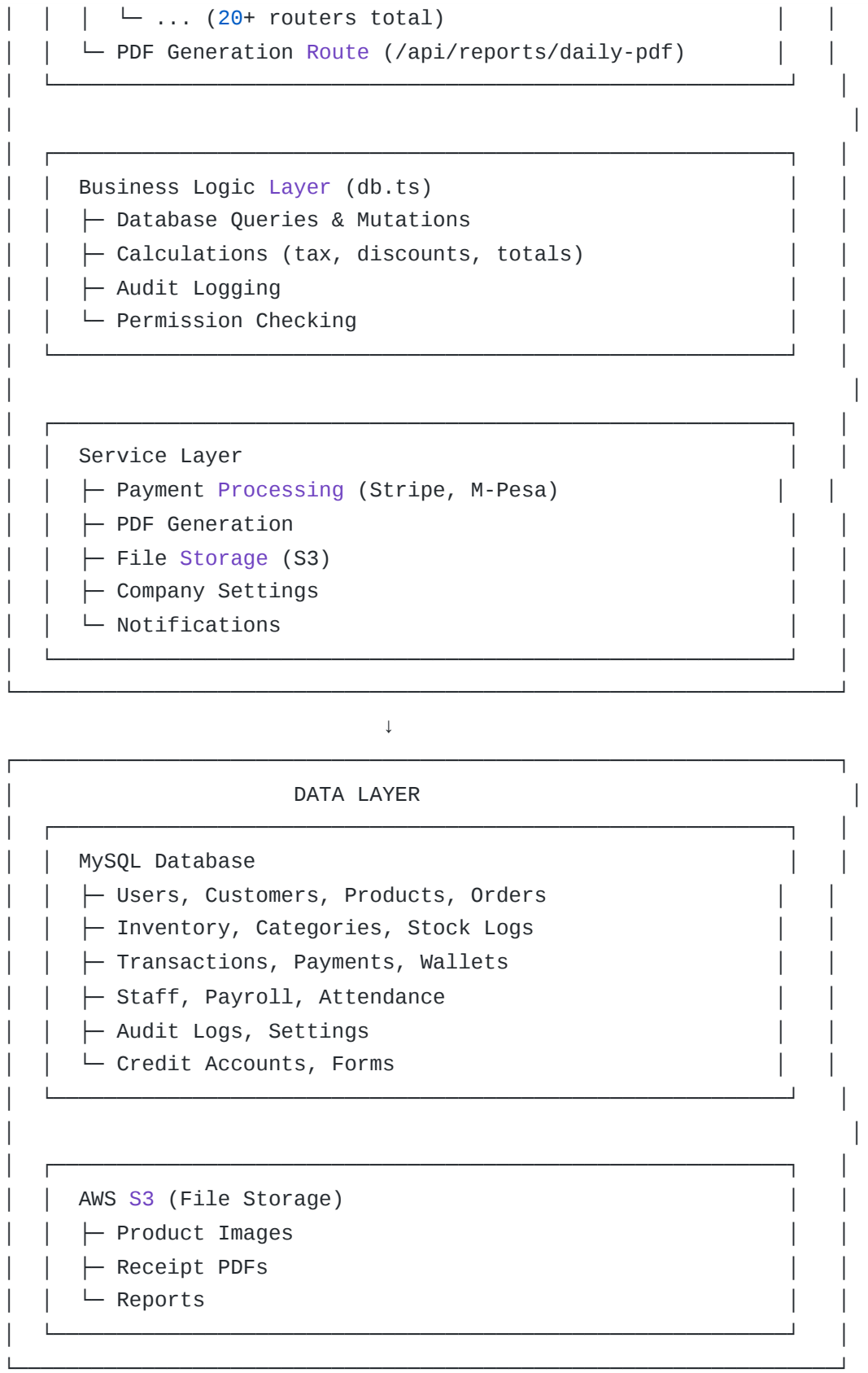
Development Tools

- **Package Manager:** pnpm
- **Code Formatter:** Prettier
- **Linter:** TypeScript compiler (tsc)
- **Database Migrations:** Drizzle Kit
- **Build Bundler:** esbuild (server), Vite (client)

System Architecture

High-Level Architecture Diagram




└─ ... (20+ routers total)
└─ PDF Generation Route (/api/reports/daily-pdf)
Business Logic Layer (db.ts)
├─ Database Queries & Mutations
├─ Calculations (tax, discounts, totals)
├─ Audit Logging
└─ Permission Checking
Service Layer
├─ Payment Processing (Stripe, M-Pesa)
├─ PDF Generation
├─ File Storage (S3)
├─ Company Settings
└─ Notifications
↓
DATA LAYER
MySQL Database
├─ Users, Customers, Products, Orders
├─ Inventory, Categories, Stock Logs
├─ Transactions, Payments, Wallets
├─ Staff, Payroll, Attendance
├─ Audit Logs, Settings
└─ Credit Accounts, Forms
AWS S3 (File Storage)
├─ Product Images
├─ Receipt PDFs
└─ Reports

Request Flow Example: Creating a Sales Order

```
1. User enters product, quantity, and customer in POS UI
2. Frontend calls: trpc.orders.create.useMutation()
3. React Query sends HTTP POST to /api/trpc/orders.create
4. Express receives request, validates tRPC payload
5. tRPC middleware calls createContext() to get user info
6. protectedProcedure checks if user is authenticated
7. Input validation (Zod schema) runs
8. createOrder() function executes in db.ts:
   - Generates unique order number
   - Calculates tax and totals
   - Inserts order into database
   - Records inventory adjustments
   - Logs audit trail
9. Response serialized with superjson (handles Date objects)
10. HTTP 200 response sent to client
11. React Query updates cache
12. UI re-renders with new order
13. User sees success toast notification
```

Database Schema

Core Tables Overview

The database consists of **15+ core tables** organized into logical domains:

1. User Management

- **users:** System users with roles and authentication
- **staffProfiles:** Employee information and contact details
- **staffEmployment:** Employment records and contract info
- **userRoles:** Custom role assignments per user

2. Product & Inventory

- **products:** Product master data (name, price, SKU, stock)

- **categories:** Product categories with colors
- **inventoryLogs:** Stock movement history
- **suppliers:** Vendor information

3. Sales & Orders

- **orders:** Sales transactions with payment info
- **orderItems:** Individual line items per order
- **orderStatusHistory:** Order status changes over time
- **payments:** Payment transaction records

4. Customers

- **customers:** Customer master data with loyalty points
- **customerWallets:** Prepaid wallet balances
- **creditAccounts:** Credit/debit accounts for customers
- **customerWalletTransactions:** Wallet transaction history

5. Operations

- **forms:** Group feeding forms (bulk orders)
- **branches:** Multi-branch support
- **servingPoints:** Serving locations within branches
- **settings:** System configuration (company info, tax rates)

6. Staff & Payroll

- **attendance:** Daily attendance records
- **payrollRecords:** Monthly payroll calculations
- **payrollDeductions:** Salary deductions
- **payrollBonuses:** Salary bonuses
- **payslips:** Generated payslips

7. Kitchen & Operations

- **kitchenStaff:** Kitchen staff assignments
- **kdsSettings:** Kitchen Display System configuration
- **orderStatusHistory:** Order progress tracking

8. Audit & Compliance

- **auditLogs:** System action audit trail
- **staffActivityLogs:** User activity tracking

Key Relationships

```
users (1) ──→ (many) orders
customers (1) ──→ (many) orders
products (1) ──→ (many) orderItems
categories (1) ──→ (many) products
customers (1) ──→ (1) customerWallets
customers (1) ──→ (1) creditAccounts
staffProfiles (1) ──→ (many) staffActivityLogs
orders (1) ──→ (many) orderStatusHistory
```

Data Types & Constraints

- **Decimal Fields:** `DECIMAL(12,2)` for currency (prices, amounts)
- **Enums:** MySQL ENUM for payment methods, order status, payment status
- **Timestamps:** `TIMESTAMP DEFAULT CURRENT_TIMESTAMP` for audit trails
- **Foreign Keys:** Referential integrity enforced at database level
- **Indexes:** Created on frequently queried fields (customerId, productId, orderNumber)

Backend Architecture

Server Entry Point (`server/_core/index.ts`)

The Express server initializes with the following sequence:

1. **Database Initialization:** `initializeDatabase()` creates tables if needed
2. **Express Setup:** Configure body parsers (50MB limit for file uploads)
3. **OAuth Routes:** Register `/api/oauth/callback` for Manus OAuth
4. **PDF Endpoint:** Special REST route for daily sales PDF generation
5. **tRPC Middleware:** Mount `/api/trpc` with all procedures
6. **Static Serving:** Vite dev server (dev) or static files (production)
7. **Port Discovery:** Find available port starting from 3000

tRPC Architecture

tRPC provides end-to-end type safety from backend to frontend:

```
// Backend: Define procedure
export const ordersRouter = router({
  create: protectedProcedure
    .input(z.object({
      customerId: z.number().optional(),
      items: z.array(z.object({ productId: z.number(), quantity: z.number()
})),
      paymentMethod: z.enum(["cash", "mpesa", "stripe", "mixed", "wallet"])
    }))
    .mutation(async ({ input, ctx }) => {
      // ctx.user is automatically available (type-safe)
      return createOrder(input, ctx.user.id);
    })
});

// Frontend: Automatically typed
const createOrderMutation = trpc.orders.create.useMutation();
createOrderMutation.mutate({
  customerId: 1,
  items: [...],
  paymentMethod: "cash"
}); // TypeScript catches errors at compile time
```

Router Organization

The backend exposes **20+ routers** organized by domain:

Router	Procedures	Purpose
auth	login, logout, me	User authentication
categories	list, create, update, delete	Product categories
products	list, create, update, delete, getById	Product management
orders	create, list, getById, updateStatus	Sales orders
customers	list, create, update, delete, getById	Customer management
reports	dashboard, sales, inventory, topProducts, allProducts	Business reports
users	list, create, updateRole, delete, deactivate, reactivate	User management
staff	createProfile, listProfiles, updateProfile, recordActivity	Staff management
payroll	createRecord, listRecords, generatePayslip, updateSettings	Payroll processing
wallet	getBalance, addFunds, spend, getTransactions	Customer wallet
credit	createAccount, getAccount, recordTransaction, updateBalance	Credit system
forms	create, list, update, updateStatus	Group feeding forms
audit	getLogs, recordLog	Audit trail
branches	list, create	Multi-branch support
suppliers	list, create, update	Supplier management
kds	getQueue, updateStatus, getSettings, updateSettings	Kitchen Display System
system	health, notifyOwner	System utilities

Middleware & Context

Each tRPC request builds a **context** containing:

```
interface Context {
  req: Express.Request;
  res: Express.Response;
  user: User | null; // Authenticated user or null
}
```

Procedure Types:

- `publicProcedure` - No authentication required
- `protectedProcedure` - Requires authenticated user
- `adminProcedure` - Requires admin role

Database Layer (`server/db.ts`)

The `db.ts` file contains **100+ database functions** organized by domain:

Pattern:

```
export async function createOrder(data: InsertOrder, userId: number) {
  const db = await getDb();

  // Validate business rules
  if (data.totalAmount < 0) throw new Error("Invalid amount");

  // Execute transaction
  const result = await db.insert(orders).values({
    ...data,
    orderNumber: generateOrderNumber(),
    createdBy: userId
  });

  // Log audit trail
  await recordAuditLog(userId, "ORDER_CREATED", { orderId: result.insertId
});

  return result;
}
```

Key Service Modules

Payment Processing (`server/mpesa.ts`)

- M-Pesa STK push initiation
- Payment status polling
- Transaction verification

PDF Generation (`server/pdf-generator.ts`)

- Daily sales report PDF
- Receipt generation
- Itemized transaction lists with pagination

File Storage (`server/storage.ts`)

- AWS S3 integration
- Presigned URLs for downloads
- Product image uploads

Company Settings (`server/companySettings.ts`)

- Branding (logo, name, address)
- Tax configuration
- Business hours

Audit Logging (`server/auditLogger.ts`)

- User action tracking
- Role assignment logging
- User creation/deletion audit

Frontend Architecture

Application Structure

```
client/
├── src/
│   ├── pages/              # Page components (one per route)
│   ├── components/         # Reusable UI components
│   ├── contexts/           # React contexts (Theme, Auth)
│   ├── hooks/              # Custom React hooks
│   ├── lib/                # Utilities (tRPC, permissions, exports)
│   ├── _core/              # Core functionality (auth, hooks)
│   ├── App.tsx             # Route definitions
│   ├── main.tsx            # React bootstrap
│   └── index.css           # Global styles
├── public/                 # Static assets (favicon, robots.txt)
└── index.html              # HTML template
```

Layout System

POSLayout (`client/src/components/POSLayout.tsx`) is the main authenticated shell:

- **Sidebar Navigation:** Collapsible menu with role-based visibility
- **Header:** Company branding, user profile, logout
- **Main Content:** Page-specific content
- **Responsive:** Collapses to mobile-friendly layout

Navigation Structure:

```
Dashboard
├─ Inventory
│  ├─ Products
│  ├─ Categories
│  └─ Stock Alerts
├─ Sales
│  ├─ Sales Transaction (POS)
│  ├─ Sales Orders
│  ├─ Customers
│  └─ Transactions
├─ Reports
│  ├─ Sales Report
│  ├─ Inventory Report
│  ├─ Customer Spending
│  └─ Transaction Reports
├─ Customer Wallet
├─ Operations
│  ├─ Group Feeding Forms
│  ├─ Credit System
│  ├─ Branches
│  └─ Suppliers
├─ Audit Trail
├─ Administration
│  ├─ User Management
│  ├─ Role Permissions
│  ├─ Custom Roles
│  ├─ Audit Logs
│  ├─ Staff Activity
│  ├─ Staff Management
│  └─ Payroll
├─ Kitchen Display
├─ Serving Display
└─ Settings
```

State Management

React Query manages server state:

```
// Fetching data
const { data: products, isLoading } = trpc.products.list.useQuery();

// Mutating data with optimistic updates
const updateMutation = trpc.products.update.useMutation({
  onMutate: async (newData) => {
    // Optimistically update cache
    await queryClient.cancelQueries({ queryKey: ["products"] });
    const previousData = queryClient.getQueryData(["products"]);
    queryClient.setQueryData(["products"], (old) => [...old, newData]);
    return { previousData };
  },
  onError: (err, newData, context) => {
    // Rollback on error
    queryClient.setQueryData(["products"], context?.previousData);
  }
});
```

Component Library

shadcn/ui provides pre-built components:

- Form inputs (Text, Select, Checkbox, Radio)
- Dialogs, Modals, Popovers
- Tables with sorting/filtering
- Cards, Badges, Buttons
- Tabs, Accordion, Collapsible
- Date pickers, Dropdowns
- Alerts, Toasts

Custom Components:

- `DashboardLayout` - Dashboard shell with sidebar
- `POSLayout` - POS shell with navigation
- `ReportPreviewModal` - PDF preview before export
- `PaymentDialog` - Payment method selection
- `ReceiptDialog` - Order receipt display

- `ProductGrid` - Product selection interface

Frontend Pages & Features

1. Dashboard (`/dashboard`)

Purpose: Executive overview of business metrics

Features:

- **KPI Cards:** Today's revenue, monthly revenue, customer count, active products, low-stock count
- **Revenue Chart:** Weekly revenue trend (Recharts area chart)
- **Low Stock Alerts:** Products below threshold with quick links
- **Payment Breakdown:** Revenue by payment method (pie chart)
- **Recent Orders:** Last 10 orders with status
- **Quick Actions:** Links to POS, inventory alerts, sales orders

Data Source: `trpc.reports.dashboard.useQuery()`

Key Metrics:

```
{
  todayRevenue: number;
  monthlyRevenue: number;
  totalCustomers: number;
  activeProducts: number;
  lowStockCount: number;
  weeklyRevenue: Array<{ date: string; revenue: number }>;
  paymentMethods: Array<{ method: string; amount: number }>;
  recentOrders: Order[];
}
```

2. Sales Transaction / POS (`/sales/pos`)

Purpose: Real-time point of sale transaction processing

Features:

- **Product Selection:** Grid view with search, category filter, image
- **Shopping Cart:** Add/remove items, quantity adjustment, running total
- **Customer Selection:** Search existing customers or add new
- **Tax Calculation:** Automatic tax based on settings
- **Payment Methods:** Cash, M-Pesa, Stripe, wallet, mixed payment
- **Receipt Generation:** Print or email receipt
- **Payment Dialog:** Handle payment processing and change calculation
- **Order Confirmation:** Success message with order number

Workflow:

1. Select category and products
2. Add items to cart with quantities
3. Select customer (optional)
4. Review subtotal, tax, total
5. Choose payment method
6. Process payment
7. Print/email receipt
8. Order marked as completed

State Management:

```
const [cart, setCart] = useState<CartItem[]>([]);
const [selectedCustomer, setSelectedCustomer] = useState<Customer | null>
(null);
const [paymentMethod, setPaymentMethod] = useState<PaymentMethod>("cash");
const [taxRate, setTaxRate] = useState(0.16); // 16% VAT
```

3. Sales Orders (`/sales/orders`)

Purpose: View and manage all sales orders

Features:

- **Order List:** Searchable, sortable table with order number, customer, amount, status, date
- **Order Details:** Click to view full order with line items
- **Status Updates:** Change order status (pending → processing → completed)
- **Payment Status:** Track payment status (pending, paid, failed, refunded)
- **Filters:** By date range, customer, payment method, status
- **Export:** Download orders as CSV/Excel

Columns:

- Order Number
- Customer Name
- Total Amount (KES)
- Payment Method
- Payment Status
- Order Status
- Date
- Actions (View, Edit, Delete)

4. Inventory (`/inventory/products`, `/inventory/categories`, `/inventory/alerts`)

Purpose: Manage product catalog and stock levels

Features:

Products Tab:

- Add new products (name, SKU, price, category, stock, image)
- Edit product details
- Delete products
- Bulk import via CSV
- Search and filter by category

- Display stock quantity and low-stock threshold

Categories Tab:

- Create product categories
- Assign colors to categories
- Edit category names
- Delete categories

Stock Alerts Tab:

- List all low-stock products
- Set custom thresholds
- Quick reorder button
- Stock history view

Data Fields per Product:

```
{
  id: number;
  name: string;
  sku: string;
  price: decimal;
  originalPrice?: decimal;
  categoryId: number;
  imageUrl?: string;
  stockQuantity: number;
  lowStockThreshold: number;
  unit: string; // "pcs", "kg", "ltr", etc.
  barcode?: string;
  isActive: boolean;
}
```

5. Customers (`/sales/customers`)

Purpose: Manage customer database and relationships

Features:

- **Customer List:** Name, phone, email, total spent, loyalty points, registration date
- **Add Customer:** Create new customer with contact info
- **Customer Details:** View spending history, wallet balance, credit account
- **Loyalty Points:** Track and redeem points
- **Spending Reports:** Customer spending trends
- **Search & Filter:** By name, phone, date range

Customer Profile:

```
{
  id: number;
  name: string;
  phone: string;
  email?: string;
  address?: string;
  loyaltyPoints: number;
  totalSpent: decimal;
  notes?: string;
  createdAt: timestamp;
  updatedAt: timestamp;
}
```

6. Reports (`/reports/sales`, `/reports/inventory`, `/reports/customer-spending`, `/reports/transactions`)

Purpose: Business analytics and reporting

Sales Report

- **Date Range Selection:** From/to date picker
- **Summary Metrics:** Total revenue, orders, average order value, VAT
- **Payment Breakdown:** Revenue by payment method
- **Product List:** All products sold with quantity and revenue
- **PDF Preview:** See formatted PDF before download
- **Export Options:** PDF, Excel, CSV

PDF Features:

- Company header with branding
- Report title and date range
- Summary section with KPIs
- Payment methods table
- Itemized products table with pagination
- Footer with company info

Excel Features:

- Multiple sheets: Summary, All Products, Payment Methods
- Formatted headers and totals
- Proper number formatting (currency, decimals)

Inventory Report

- Stock levels by category
- Low-stock warnings
- Stock movement history
- Reorder recommendations

Customer Spending Report

- Customer spending trends over time
- Top customers by spending
- Payment method breakdown per customer
- Loyalty points earned

Transaction Reports

- All transactions with details
- Filter by date, customer, payment method
- Export transaction history

7. Customer Wallet (`/wallet`)

Purpose: Manage prepaid customer wallet system

Features:

- **Wallet Balance:** Current balance per customer
- **Add Funds:** Deposit money into wallet
- **Spend from Wallet:** Use wallet balance for purchases
- **Transaction History:** All wallet transactions with dates
- **Wallet Statements:** Export wallet activity

Workflow:

1. Customer selects “Add to Wallet” option
2. Choose amount and payment method
3. Process payment
4. Funds added to wallet balance
5. Use wallet balance at checkout

8. Credit System (`/credit`)

Purpose: Manage customer credit accounts

Features:

- **Credit Accounts:** Create credit account for customer
- **Credit Limit:** Set maximum credit amount
- **Record Transactions:** Track credit purchases and payments
- **Credit Balance:** Current balance owed
- **Payment Tracking:** Record payments received
- **Aging Report:** Overdue accounts

Credit Account:

```
{
  id: number;
  customerId: number;
  totalCredit: decimal;
  totalPaid: decimal;
  balance: decimal;
  status: "active" | "suspended" | "closed";
  createdAt: timestamp;
}
```

9. Group Feeding Forms (`/forms`)

Purpose: Manage bulk order forms for group feeding

Features:

- **Create Form:** New group feeding form with items and quantities
- **Form Status:** Draft, submitted, completed
- **Track Spending:** Total amount spent on form
- **Item List:** Products ordered with quantities
- **Completion:** Mark form as completed

10. Branches (`/branches`)

Purpose: Multi-branch management

Features:

- **Branch List:** All branches with location info
- **Add Branch:** Create new branch
- **Branch Details:** Address, phone, manager
- **Serving Points:** Sub-locations within branch
- **Branch-Specific Reports:** Sales per branch

11. Suppliers (`/suppliers`)

Purpose: Supplier management

Features:

- **Supplier List:** Name, contact, products supplied
- **Add Supplier:** Create new supplier
- **Contact Info:** Phone, email, address
- **Products:** Which products they supply
- **Purchase History:** Orders from supplier

12. Kitchen Display System (`/kitchen`)

Purpose: Real-time order queue for kitchen staff

Features:

- **Order Queue:** Pending orders in real-time
- **Order Details:** Items, quantities, special instructions
- **Status Updates:** Mark items as preparing, ready, completed
- **Time Tracking:** How long order has been pending
- **Audio Alerts:** Notify kitchen of new orders
- **Display Refresh:** Auto-refresh every 5 seconds

Order Status Flow:

```
Pending → Preparing → Ready → Completed
```

13. Serving Display (`/serving`)

Purpose: Customer-facing order status display

Features:

- **Order Status:** Show customers their order status

- **Queue Position:** Where their order is in the queue
- **Estimated Time:** When order will be ready
- **Order Number:** Large display of order number

14. User Management (`/users`)

Purpose: System user administration

Features:

- **User List:** All system users with roles
- **Create User:** Add new user with role assignment
- **Role Assignment:** Change user role
- **Deactivate/Reactivate:** Disable/enable user access
- **Delete User:** Remove user from system
- **Activity Logs:** Track user actions

User Roles:

- Admin - Full system access
- Manager - Sales, inventory, reports, staff
- Supervisor - Sales, inventory, reports
- Cashier - Sales transactions only
- Waiter - Sales, customers
- Inventory Manager - Inventory only
- Kitchen Staff - Kitchen display only

15. Staff Management (`/staff-management`)

Purpose: Employee management and tracking

Features:

- **Staff Profiles:** Employee information
- **Attendance:** Daily attendance tracking

- **Activity Logs:** Track staff actions
- **Performance:** Metrics and KPIs
- **Employment Records:** Hire date, contract info

16. Payroll (`/payroll`)

Purpose: Employee payroll processing

Features:

- **Payroll Records:** Monthly payroll calculations
- **Deductions:** Salary deductions (tax, insurance, etc.)
- **Bonuses:** Performance bonuses
- **Payslips:** Generate and distribute payslips
- **Attendance:** Automatic calculation based on attendance
- **Casual Laborers:** Hourly rate calculation

Payroll Calculation:

```
Gross Salary = Base Salary + Bonuses
Deductions = Tax + Insurance + Other
Net Salary = Gross Salary - Deductions
```

17. Audit Logs (`/audit-logs`)

Purpose: System audit trail and compliance

Features:

- **Action Log:** All user actions with timestamp
- **User Info:** Who performed the action
- **Action Type:** What was done (CREATE, UPDATE, DELETE)
- **Details:** What changed
- **Search & Filter:** By user, date, action type
- **Export:** Download audit trail

Logged Actions:

- User login/logout
- Product create/update/delete
- Order creation
- Payment processing
- User role changes
- Settings changes

18. Role Permissions (`/role-permissions`)

Purpose: Manage role-based access control

Features:

- **Permission Matrix:** View all roles and their permissions
- **Feature Access:** Which features each role can access
- **Procedure Access:** Which API procedures each role can call
- **Custom Roles:** Create custom role with specific permissions
- **Role Assignment:** Assign roles to users

Permission Model:

```typescript
interface RolePermissions {
  dashboard: boolean;
  sales: boolean;
  inventory: boolean;
  reports: boolean;
  wallet: boolean;
  forms: boolean;
  credit: boolean;
  auditLogs: boolean;
  branches: boolean;
  suppliers: boolean;
  userManagement: boolean;
  staffManagement: boolean;
  payroll: boolean;
  kds: boolean;
  servingDisplay: boolean;
  settings: boolean;
  rolePermissions: boolean;
}
```

19. Settings (`/settings`)

Purpose: System configuration

Features:

- **Company Info:** Name, logo, address, phone, email, website, tax ID
- **Tax Settings:** Tax rate (VAT), tax ID
- **Payment Methods:** Enable/disable payment methods
- **Receipt Settings:** Print options, email settings
- **Business Hours:** Opening/closing times
- **Notification Preferences:** Email alerts, SMS alerts

20. Transactions (`/sales/transactions`)

Purpose: View all payment transactions

Features:

- **Transaction List:** All transactions with details
- **Transaction Details:** Payment method, amount, date, status
- **Search & Filter:** By customer, date, amount, method
- **Reconciliation:** Match transactions to orders
- **Export:** Download transaction history

Authentication & Authorization

OAuth 2.0 Flow

Manus OAuth Integration:

1. **Login Initiation:** User clicks “Login” button
2. **Redirect to Manus:** Frontend redirects to Manus OAuth portal
3. **User Authentication:** User enters credentials at Manus
4. **Authorization:** Manus redirects back to `/api/oauth/callback`
5. **Token Exchange:** Backend exchanges auth code for access token
6. **Session Creation:** Backend creates session cookie
7. **Redirect to App:** User redirected to dashboard
8. **Authenticated Requests:** All subsequent requests include session cookie

Frontend Auth Hook

```
// client/src/_core/hooks/useAuth.ts
const { user, loading, isAuthenticated } = useAuth();

// user: { id, name, email, role, ... } | null
// loading: boolean (true while checking auth status)
// isAuthenticated: boolean (true if user is logged in)
```

Backend Auth Context

```
// server/_core/context.ts
interface Context {
  user: User | null; // Null if not authenticated
  req: Express.Request;
  res: Express.Response;
}
```

Procedure Protection

```
// Public procedure - no auth required
publicProcedure.query(() => getCategories());

// Protected procedure - requires login
protectedProcedure.mutation(async ({ ctx }) => {
  if (!ctx.user) throw new TRPCError({ code: "UNAUTHORIZED" });
  // Proceed with user-specific logic
});

// Admin procedure - requires admin role
adminProcedure.mutation(async ({ ctx }) => {
  if (ctx.user?.role !== "admin") throw new TRPCError({ code: "FORBIDDEN"
});
  // Proceed with admin-only logic
});
```

Role-Based Access Control (RBAC)

Frontend Permission Checking:

```
// client/src/lib/rolePermissions.ts
const canAccessFeature = (role: UserRole, feature: string): boolean => {
  return rolePermissionsMap[role][feature] === true;
};

// Usage in navigation
{canAccessFeature(user.role, "payroll") && (
  <Link href="/payroll">Payroll</Link>
)}
```

Backend Permission Checking:

```
// server/permissions.ts
export const requirePermission = (permission: string) => {
  return async ({ ctx, next }) => {
    if (!ctx.user) throw new TRPCError({ code: "UNAUTHORIZED" });
    if (!canAccessFeature(ctx.user.role, permission)) {
      throw new TRPCError({ code: "FORBIDDEN" });
    }
    return next({ ctx });
  };
};

// Usage in procedure
protectedProcedure
  .use(requirePermission("payroll"))
  .mutation(async ({ input }) => {
    // Only users with payroll permission can access
  });
```

Key Features & Workflows

1. Complete Sales Transaction Workflow

Scenario: Customer purchases items at POS

```
1. Cashier opens POS page (/sales/pos)
2. System loads active products and categories
3. Cashier selects category (e.g., "Beverages")
4. Products display in grid
5. Cashier clicks product and enters quantity
6. Item adds to cart with subtotal
7. Cashier can add more items
8. Cashier selects customer (optional)
9. System calculates:
   - Subtotal (sum of items)
   - Tax (subtotal × tax rate)
   - Total (subtotal + tax)
10. Cashier selects payment method:
    - Cash: Enter amount received, calculate change
    - M-Pesa: Enter phone, initiate STK push
    - Stripe: Enter card details
    - Wallet: Deduct from customer wallet
    - Mixed: Combine multiple methods
11. System processes payment
12. Order created in database
13. Receipt generated and printed/emailed
14. Inventory adjusted (stock decremented)
15. Customer wallet/credit updated if applicable
16. Audit log recorded
17. Order marked as completed
18. Cart cleared for next transaction
```

Database Changes:

- New order inserted into `orders` table
- Order items inserted into `orderItems` table
- Product stock decremented in `products` table
- Payment recorded in `payments` table
- Customer wallet updated if applicable
- Audit log entry created

2. Inventory Management Workflow

Scenario: Low stock alert and reorder

```
1. System monitors stock levels
2. Product stock falls below threshold
3. Low-stock alert appears on dashboard
4. Manager views inventory alerts page
5. System shows low-stock products
6. Manager clicks "Reorder" button
7. System creates purchase order for supplier
8. Supplier delivers stock
9. Manager updates stock quantity
10. Inventory log records stock increase
11. Alert clears from dashboard
```

3. Customer Credit Workflow

Scenario: Customer purchases on credit

```
1. Customer requests credit purchase
2. Cashier checks customer credit account
3. Credit account shows available balance
4. Cashier processes sale with "Credit" payment method
5. Order created with payment_status = "pending"
6. Credit balance decremented
7. Customer receives invoice
8. Customer makes payment later
9. Manager records payment in credit system
10. Credit balance updated
11. Payment status changed to "paid"
```

4. Payroll Processing Workflow

Scenario: Monthly payroll calculation

```
1. HR manager opens Payroll page
2. Selects month to process
3. System retrieves:
   - Employee records
   - Attendance data
   - Base salaries
   - Deductions
   - Bonuses
4. System calculates:
   - Days worked from attendance
   - Gross salary (base + bonuses)
   - Deductions (tax, insurance)
   - Net salary
5. Manager reviews calculations
6. Manager approves payroll
7. Payslips generated
8. Payroll status marked as "processed"
9. Payslips sent to employees
10. Payment recorded in system
```

5. Multi-Payment Order Workflow

Scenario: Customer pays with cash + wallet

```
1. Order total: KES 1,000
2. Customer pays KES 600 in cash
3. Remaining: KES 400
4. Customer uses wallet for KES 400
5. System processes:
   - Record KES 600 cash payment
   - Deduct KES 400 from wallet
   - Mark order as fully paid
6. Payment method recorded as "mixed"
7. Receipt shows both payment methods
```

6. Kitchen Display System Workflow

Scenario: Order preparation tracking

```
1. Cashier completes sale
2. Order sent to kitchen display
3. Kitchen staff sees new order
4. Audio alert notifies kitchen
5. Kitchen staff marks order as "Preparing"
6. Kitchen staff prepares items
7. Kitchen staff marks order as "Ready"
8. Serving staff notified
9. Serving staff delivers order
10. Serving staff marks order as "Completed"
11. Order removed from queue
```

Deployment & Environment

Environment Variables

Required Environment Variables:

```
# Database
DATABASE_URL=mysql://user:password@localhost:3306/kenya_pos

# OAuth
VITE_APP_ID=your_manus_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://oauth.manus.im

# JWT
JWT_SECRET=your_jwt_secret_key

# Stripe
STRIPE_SECRET_KEY=sk_live_...
STRIPE_PUBLISHABLE_KEY=pk_live_...

# M-Pesa
MPESA_CONSUMER_KEY=your_consumer_key
MPESA_CONSUMER_SECRET=your_consumer_secret
MPESA_PASSKEY=your_passkey

# AWS S3
AWS_ACCESS_KEY_ID=your_access_key
AWS_SECRET_ACCESS_KEY=your_secret_key
AWS_REGION=us-east-1
AWS_S3_BUCKET=your_bucket_name

# Owner Info
OWNER_NAME=Your Name
OWNER_OPEN_ID=your_open_id

# Manus APIs
BUILT_IN_FORGE_API_URL=https://api.manus.im
BUILT_IN_FORGE_API_KEY=your_api_key
VITE_FRONTEND_FORGE_API_URL=https://api.manus.im
VITE_FRONTEND_FORGE_API_KEY=your_frontend_key
```

Build & Deployment

Development:

```
pnpm install
pnpm dev
# Runs on http://localhost:3000
```

Production Build:

```
pnpm build
# Creates dist/ folder with compiled server and client
```

Production Start:

```
NODE_ENV=production pnpm start
# Runs compiled server
```

Database Migrations

Generate Migration:

```
pnpm drizzle-kit generate
# Creates SQL migration file
```

Apply Migration:

```
pnpm drizzle-kit migrate
# Applies pending migrations to database
```

Deployment Platforms

Manus WebDev (Built-in):

- Automatic deployment on git push
- Custom domain support

- SSL certificates included
- Scalable infrastructure

Alternative Platforms:

- Railway
- Render
- Vercel (frontend only)
- AWS (full stack)

Performance Considerations

Frontend Optimization

- **Code Splitting:** Routes lazy-loaded with Vite
- **Image Optimization:** Product images served from S3 with CDN
- **Caching:** React Query caches API responses
- **Pagination:** Large lists paginated (20-50 items per page)
- **Debouncing:** Search inputs debounced to reduce API calls
- **Memoization:** Components memoized to prevent unnecessary re-renders

Backend Optimization

- **Database Indexes:** Created on frequently queried fields
- **Query Optimization:** Drizzle ORM generates efficient SQL
- **Connection Pooling:** MySQL connection pool configured
- **Caching:** Company settings cached in memory
- **Batch Operations:** Multiple items processed in single transaction
- **Async Processing:** Long-running tasks (PDF generation) handled asynchronously

Scalability

- **Horizontal Scaling:** Stateless Express server can be replicated
- **Database Replication:** MySQL can be replicated across servers
- **CDN:** Static assets served from CDN
- **Load Balancing:** Multiple server instances behind load balancer
- **Message Queue:** Long-running tasks queued for background processing

---

Security Considerations

Authentication & Authorization

- **OAuth 2.0:** Industry-standard authentication
- **Session Cookies:** Secure, httpOnly, sameSite cookies
- **JWT:** Optional JWT for API access
- **RBAC:** Role-based access control at frontend and backend
- **Permission Checking:** Every procedure validates user permissions

Data Protection

- **HTTPS:** All traffic encrypted in transit
- **SQL Injection Prevention:** Drizzle ORM uses parameterized queries
- **XSS Prevention:** React escapes user input by default
- **CSRF Protection:** Express middleware validates CSRF tokens
- **Input Validation:** Zod schemas validate all inputs

Audit & Compliance

- **Audit Logs:** All user actions logged with timestamp
- **Data Retention:** Logs retained for compliance period
- **User Activity Tracking:** Staff activity logged

- **Payment Audit:** All payment transactions logged
- **Compliance Reports:** Export audit logs for compliance

Future Enhancements

Planned Features

1. **Real-time Notifications:** WebSocket-based order notifications
2. **Mobile App:** React Native mobile app for staff
3. **Advanced Analytics:** Machine learning for sales forecasting
4. **Inventory Forecasting:** Predict stock needs
5. **Customer Analytics:** Spending patterns and recommendations
6. **Integration APIs:** Third-party integrations (accounting software, etc.)
7. **Multi-currency:** Support for multiple currencies
8. **Loyalty Program:** Advanced loyalty points system
9. **Franchise Support:** Multi-franchise management
10. **Offline Mode:** Offline POS capability with sync

Technical Debt

- Migrate to TypeScript strict mode
- Add comprehensive unit tests
- Implement E2E tests with Cypress
- Optimize database queries
- Refactor large components
- Add API rate limiting

Conclusion

KenPOS is a comprehensive, production-ready point of sale system built with modern technologies and best practices. The architecture balances performance, scalability, and maintainability while providing a rich feature set for restaurant and retail operations.

The system demonstrates:

- **Type Safety:** End-to-end TypeScript with tRPC
- **Security:** OAuth, RBAC, audit logging
- **Scalability:** Horizontal scaling capability
- **User Experience:** Responsive UI with real-time updates
- **Business Logic:** Complex workflows and calculations
- **Compliance:** Comprehensive audit trails

Document Version: 1.0
Last Updated: May 4, 2026
Maintained By: Development Team